

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Searching

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

Even a single day does not pass without we searching for something in our day to day life. Might be car keys, books, pen, mobile charger and what not. Same is the life of a computer. There is so much data stored in it and whenever user asks for some data, computer has to search it's memory to look for the data and make it available to the user. And the computer has it's own techniques to search through it's memory in a faster way.

Why Searching?

We often need to find one particular item of data amongst many hundreds, thousands, millions or more. For example, if one wants to find someone's phone number in your phone, or a particular business's address from address book.

A searching algorithm can be used to find the data without spending much time. What if you want to write a program to search a given integer in an array of integers? Two popular algorithms available:

Linear Search and Binary Search

Linear Search

A linear search, also known as a sequential search, is a method of finding an element within a collection. It checks each element of the list sequentially until a match is found or the whole list has been searched. It returns the position of the element in the array, else it return -1. Linear Search is applied on unsorted or unordered collection of elements.

Coding Example_1: Implementation of Linear Search

```
#include<stdio.h>
int linear_search(int *a, int n, int key);
int main()
{
    int a[100] = {22,55,77,99,25,90,100};
    int n = 7;

    int key = 100;
```

```
int res=linear_search(a,n,key);
if(res==-1)
    printf("%d not found\n",key);
else
    printf("%d found at pos %d\n",key,res);

key=177;
res=linear_search(a,n,key);
if(res==-1)
    printf("%d not found\n",key);
else
    printf("%d found at pos %d\n",key,res);
}

int linear_search(int *a, int n, int key)
{
    for(int i = 0; i < n; i++)
    {
        if(a[i]==key)
        {
            return i;
        }
    }
    return -1;
}
```

For practice: To avoid multiple return statements, linear search may also be coded as:

```
int linear_search(int *a, int n, int key)
{
    int i;
    int found = 0;
    int pos = -1;
    for(i = 0; i < n; i++)
    {
        if(a[i]==key && found == 0)
        {
            found = 1;
            pos = i;
        }
    }
    return pos;
}
```

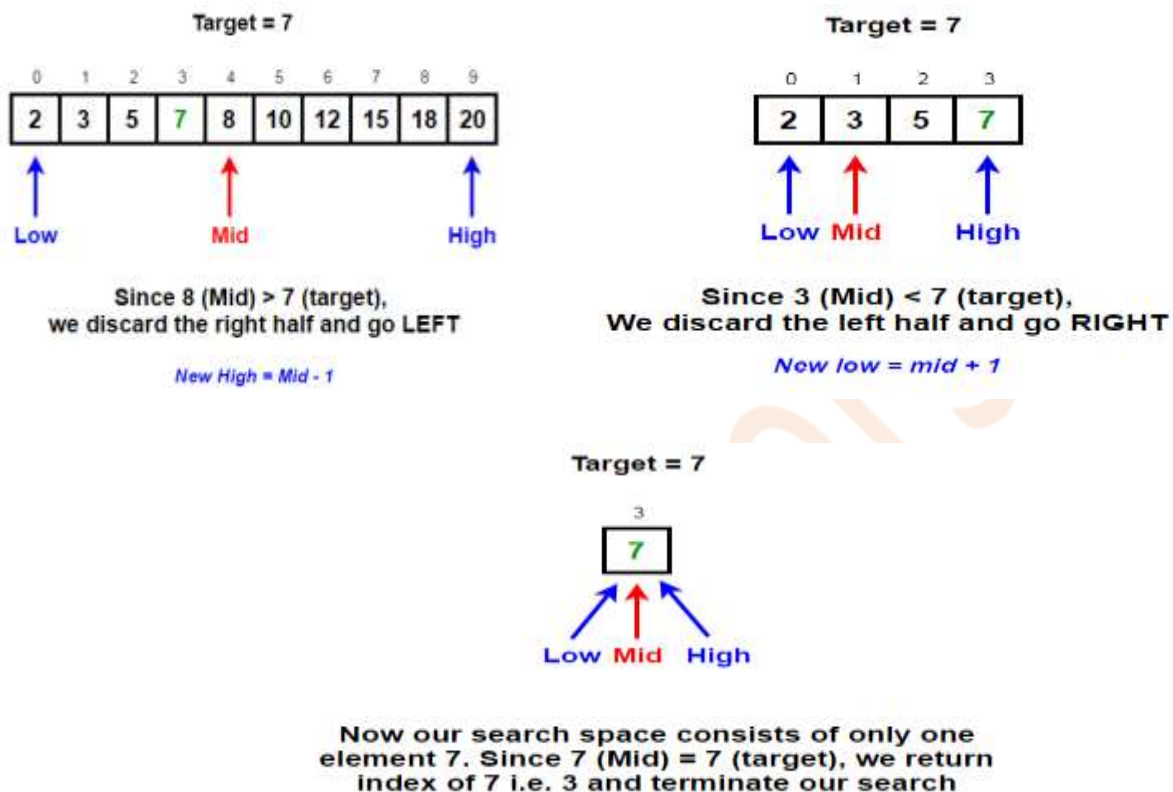
Binary Search

Binary Search is used to search an element in **a sorted array**. The following steps are applied to search an element.

1. Start by comparing the element to be searched with the element in the middle of the array.
2. If we get a match, we return the index of the middle element.
3. If we do not get a match, we check whether the element to be searched is lesser or greater in value than the middle element.
4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element(as the array is sorted, hence on the right, we will have all the numbers greater than the middle number), and start again from the step 1.
5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search Representation

Example: Take an input array by name `int arr = {2, 3, 5, 7, 8, 10, 12, 15, 18, 20}`; and **target or key** to be searched is 7.



Note: Between 2,3,5,7 and 7, write diagrams for missing two iterations. Drawing these two iterations might help you to understand better.

Coding Example_2: Implementation of Binary Search (iterative & recursive) on an array of integers

`/* Binary search: works on sorted collection of elements.`

`Time required to access each element in the collection must be same - constant time`

`*/`

`#include<stdio.h>`

`//Iterative solution`

`int binarySearchIter(int a[],int low,int high,int key)`

`{`

```
while(low<=high)
{
    int mid = (low+high)/2;
    if(a[mid]==key)
        return mid;
    else if(key<a[mid])
        high = mid-1;
    else
        low = mid+1;
}
return -1;
}

// Recursive solution
int binarySearchRec(int a[],int low,int high,int key)
{
    if(low > high) // base condition
        return -1;
    else
    {
        int mid = (low+high)/2;
        if(a[mid]==key)
        {
            return mid;
        }
        else if(key<a[mid])
            return binarySearchRec(a,low,mid-1,key);
        else
            return binarySearchRec(a,mid+1,high,key);
    }
}
```

Happy Searching!!